

Módulo 12 – Capítulo 07 – Sección 01

Framework de evaluación end-to-end: métricas del RAG, del agente y del sistema completo

La evaluación de un sistema de IA en producción no termina con el deploy — comienza con él. Los capítulos anteriores establecieron la evaluación como parte del ciclo de desarrollo: RAGAS en el pipeline CI/CD (Capítulo 6), tests de comportamiento del agente sobre el golden dataset (Capítulo 4), red teaming de seguridad (Capítulo 5). El Capítulo 7 consolida todas esas dimensiones en un framework de evaluación continua que opera en producción, detecta degradaciones de calidad que el golden dataset estático no puede capturar, y produce el input cuantitativo que orienta las decisiones de mejora del equipo.

El framework opera en tres capas que miden dimensiones distintas e irreducibles entre sí. Un sistema con excelente calidad RAG (faithfulness alto) puede tener tasa de task completion baja si el agente no sabe cuándo llamar las herramientas. Un sistema con buena task completion puede tener latencia P95 inaceptable si el agente hace demasiadas iteraciones. Un sistema con todas las métricas de calidad en verde puede tener costo por petición que excede el constraint si las queries de producción son más complejas que las del golden dataset. La evaluación end-to-end mide el sistema como un todo — la interacción entre capas, no solo las partes.

La **capa de evaluación RAG** mide la calidad de recuperación y generación usando RAGAS sobre dos conjuntos de datos complementarios. El conjunto offline es el golden dataset de 200 pares anotados, usado en cada deploy del CI/CD y en evaluaciones semanales completas. El conjunto online es el 5% del tráfico real de producción muestreado aleatoriamente: las queries reales de los usuarios, con las respuestas generadas por el sistema, evaluadas automáticamente por RAGAS con el mismo pipeline del CI/CD. La evaluación online captura distribuciones de queries que el golden dataset no cubre — preguntas que los usuarios reales formulan de formas que ningún ingeniero anticipó durante la construcción del dataset. Cuando la evaluación online muestra un gap significativo respecto a la evaluación offline (faithfulness online = 0.80 vs faithfulness offline = 0.87), esa diferencia señala que el golden

dataset no representa bien la distribución real de producción y necesita expandirse con los casos de producción donde el sistema falla.

La **capa de evaluación agéntica** mide dimensiones que RAGAS no captura. La task completion rate (¿el agente entregó una respuesta con documentos citados?) mide si el agente converge a una respuesta útil. La hallucination rate evaluada por LLM-as-judge (¿la respuesta contiene afirmaciones que no están en los documentos citados?) complementa el faithfulness de RAGAS con un juicio más contextual. La iterations per task distribution (¿cuántas llamadas a herramientas necesita el agente por pregunta?) revela si el razonamiento del agente es eficiente o si está en bucles que aumentan la latencia y el costo. La tool call accuracy (¿el agente usó la herramienta correcta para cada subtarea?) evalúa si el system prompt describe correctamente las herramientas disponibles.

La **capa de evaluación de sistema** mide rendimiento, costo y disponibilidad. La latencia P50/P95/P99 con OpenTelemetry por etapa del pipeline permite identificar el cuello de botella que causa una degradación del SLA. El costo por petición desagregado por componente (embedding + reranking + LLM + infraestructura) permite identificar qué componente está causando un incremento de costo anómalo. El error rate por tipo de error (errores de dependencia externa vs errores de validación vs errores del agente) permite priorizar qué tipo de error requiere atención inmediata.

Métricas por capa del framework de evaluación

- **Capa RAG (offline):** RAGAS faithfulness, answer_relevance, context_precision, context_recall sobre golden dataset de 200 pares — evaluado en cada deploy del CI/CD y semanalmente en producción.
- **Capa RAG (online):** las mismas métricas RAGAS sobre el 5% del tráfico real de producción muestreado aleatoriamente — captura distribuciones de producción no representadas en el golden dataset.
- **Capa agéntica:** task_completion_rate, hallucination_rate (LLM-as-judge), iterations_per_task (distribución P50/P90), tool_call_accuracy, task_accuracy_rate (evaluación completa sobre sample del 5%).
- **Capa de sistema:** latencia P50/P95/P99 por etapa y end-to-end (OpenTelemetry), throughput req/s, costo_por_peticion USD (desagregado), error_rate por tipo, disponibilidad (uptime_percent en ventana de 30 días).
- **Correlación cross-layer:** alertas que correlacionan caída de context_precision con aumento de hallucination_rate — una degradación en retrieval se propaga a la calidad de

la respuesta final.

Nota del Arquitecto: La evaluación online sobre el 5% del tráfico real es el mecanismo más valioso del framework — y el más delicado de implementar correctamente. El 5% de muestreo debe ser aleatorio y no sesgado (no solo las queries más cortas o las más frecuentes). La evaluación online usa el mismo pipeline RAGAS que la offline, pero las "respuestas esperadas" no existen — no hay golden answers para queries de producción nuevas. Por eso la evaluación online calcula *faithfulness* y *answer_relevance* (métricas que no requieren respuesta esperada) pero no *context_recall* (que requiere conocer qué información es necesaria para responder). Esta limitación debe entenderse al interpretar los resultados: una caída de *faithfulness* en la evaluación online es una señal de alarma; una caída de *context_recall* en la evaluación online simplemente no es calculable.

El framework de evaluación end-to-end no es un conjunto de scripts que se ejecutan una vez — es la infraestructura de aprendizaje continuo del sistema. Cada semana de operación en producción produce datos de evaluación que el equipo puede analizar para identificar patrones de fallo, priorizar mejoras y tomar decisiones de arquitectura con evidencia. El Capítulo 7 continúa desarrollando cada capa del framework con el detalle necesario para implementarla.

Para recordar: Un framework de evaluación end-to-end mide el sistema completo, no solo las partes — una mejora en context precision que aumenta la latencia P95 por encima del SLA no es una mejora aceptable para producción.

Módulo 12 – Capítulo 07 – Sección 02

Golden dataset de evaluación: construcción, anotación y mantenimiento como práctica de ingeniería

El golden dataset es el activo más valioso del framework de evaluación — y uno de los más subestimados en términos del esfuerzo que requiere construirlo bien. Un golden dataset de baja calidad (preguntas ambiguas, respuestas esperadas incorrectas, chunks relevantes mal identificados) produce métricas RAGAS que no correlacionan con la calidad real del sistema: el sistema puede tener faithfulness de 0.90 en el golden dataset y comportarse de forma deficiente en producción si el golden dataset no representa bien la distribución de queries reales. La inversión de tiempo en construir y mantener un golden dataset de alta calidad es la inversión con mayor retorno de todo el framework de evaluación.

La construcción del golden dataset sigue un proceso de tres fases. La **fase de generación de preguntas** produce 200 preguntas representativas del caso de uso real mediante muestreo estratificado. La estratificación garantiza que el dataset cubre los tipos de preguntas que el sistema recibirá en producción: 30% preguntas factuales simples (¿Cuál es la URL del endpoint de ingesta de la API?), 40% preguntas que requieren correlacionar información de múltiples documentos (¿Cuáles son las diferencias entre el procedimiento de deploy del servicio de pagos y el del servicio de usuarios?), y 30% preguntas sobre procedimientos paso a paso (¿Cuál es el proceso de rollback de la base de datos en caso de migración fallida?). Las preguntas se generan inicialmente por ingenieros con conocimiento del dominio; una segunda pasada con GPT-4o como asistente de generación produce variantes de las preguntas originales con reformulaciones distintas, expandiendo la cobertura de variaciones léxicas.

La **fase de anotación** produce la respuesta esperada y los chunk IDs que la soportan para cada pregunta. El anotador (un ingeniero que conoce el dominio) formula la respuesta correcta basándose en su conocimiento, luego busca en el corpus los chunks que contienen la información necesaria para respaldar esa respuesta, y los registra como `supporting_chunks: [chunk_id_1, chunk_id_2]`. Cada par incluye un `confidence_score` del anotador (1-5) que indica cuán seguro está de que la respuesta y los chunks son correctos — los pares con confidence < 3 se marcan para revisión prioritaria en la fase de revisión cruzada.

La **fase de revisión cruzada** es la más crítica y la más frecuentemente abreviada bajo presión de tiempo. Un segundo anotador (idealmente alguien que no participó en la generación) revisa cada par de forma ciega — sin ver las anotaciones del primer anotador — y produce su propia respuesta y sus propios chunk IDs. Luego se calcula el **Inter-Annotator Agreement (IAA)**: qué proporción de pares produce respuestas equivalentes entre los dos anotadores. Un IAA inferior al 80% es una señal de problemas de calidad en el dataset — indica que las preguntas son ambiguas, que los criterios de anotación no están suficientemente claros, o que el dominio tiene ambigüedades genuinas que el sistema tendrá dificultades para resolver. Los pares con discrepancia entre anotadores se discuten en una sesión de resolución de consenso; si no hay consenso, el par se descarta en lugar de introducir noise en el dataset.

El mantenimiento del golden dataset en el tiempo es una práctica de ingeniería que los equipos frecuentemente descuidan después de la construcción inicial. Tres situaciones requieren actualizar el dataset. La primera es la **expansión del corpus**: cuando se ingesta un nuevo tipo de documento o un nuevo corpus de conocimiento, se deben añadir preguntas que cubran el nuevo contenido — de lo contrario, el dataset evalúa solo la parte del sistema que ya estaba cubierta, produciendo un optimismo falso sobre la calidad del sistema completo. La segunda es la **obsolescencia del contenido**: cuando los documentos en la base de conocimiento se actualizan (un runbook que cambia el procedimiento de recuperación), las preguntas que los referencian deben actualizarse para reflejar el nuevo contenido — de lo contrario, el sistema fallará esas preguntas en producción aunque el golden dataset diga que las responde correctamente. La tercera es la **ampliación de cobertura de fallos**: cuando el análisis de la evaluación online revela tipos de preguntas donde el sistema falla consistentemente, esos tipos deben añadirse al dataset para convertir los fallos de producción en criterios de evaluación explícitos.

Proceso de construcción del golden dataset

- **Generación estratificada**: 30% factual simple, 40% multi-documento, 30% procedimental; generación inicial por ingenieros del dominio + variantes léxicas asistidas por GPT-4o; 200 pares totales.
- **Anotación con confidence**: respuesta esperada + `supporting_chunks` (lista de `chunk_ids`) + `confidence_score` (1-5) por anotador; pares con confidence < 3 marcados para revisión prioritaria.

- **Inter-Annotator Agreement:** IAA calculado como proporción de pares donde los dos anotadores producen respuestas equivalentes; umbral mínimo de 80% para considerar el dataset de calidad aceptable; pares con discrepancia a consenso o descarte.
- **Metadatos por par:** `difficulty` (easy/medium/hard), `requires_multi_doc` (boolean), `domain_area` (runbook/adr/api-spec/doc), `last_reviewed` (fecha), `annotator_id` para trazabilidad, `status` (active/deprecated/review_needed).
- **División train/test:** 160 pares para evaluación continua (CI/CD + evaluaciones semanales), 40 pares como test set reservado exclusivamente para comparaciones entre versiones mayores del sistema.
- **Mantenimiento:** revisión trimestral para obsolescencia de contenido; expansión cuando se añaden nuevas fuentes al corpus; ampliación con casos de producción donde el sistema falla consistentemente.

Nota del Arquitecto: *El Inter-Annotator Agreement es la métrica de calidad del golden dataset que más información produce y que casi nunca se calcula. En el primer proyecto donde calculé el IAA, fue del 68% — significativamente inferior al umbral del 80%. El análisis de las discrepancias reveló que el 40% de las preguntas eran ambiguas (dos ingenieros expertos del dominio las interpretaban de forma diferente), el 30% tenían respuestas correctas distintas (ambas válidas según perspectiva), y el 30% restante eran preguntas donde uno de los anotadores tenía más contexto que el otro. Esa información fue invaluable: reformulamos las preguntas ambiguas, descartamos las que tenían respuestas válidas múltiples, y subimos el IAA al 88% antes de usar el dataset para evaluación. El dataset anterior habría producido métricas de faithfulness que no correspondían a la calidad real del sistema.*

El golden dataset bien construido con IAA > 80%, división train/test, metadatos completos y proceso de mantenimiento documentado es el instrumento que hace que las métricas de evaluación del sistema sean confiables y comparables a lo largo del tiempo. El Capítulo 7 continúa con las métricas de calidad que el dataset permite calcular.

Para recordar: El golden dataset es el artefacto más valioso del framework de evaluación — su calidad, incluido el Inter-Annotator Agreement, determina la confianza en las métricas RAGAS, y debe mantenerse actualizado cuando el dominio de conocimiento evoluciona.

Módulo 12 – Capítulo 07 – Sección 03

Evaluación de calidad: interpretación operativa de faithfulness, relevance y completeness

El Capítulo 3 introdujo RAGAS como herramienta de validación durante la implementación del pipeline RAG: las métricas sirvieron como guía para elegir los parámetros de chunking, seleccionar el modelo de embedding y calibrar el threshold del reranker. En producción, las mismas métricas RAGAS cumplen un rol diferente: son el instrumento de interpretación operativa que permite al equipo detectar degradaciones, diagnosticar sus causas y priorizar las mejoras correctas. La perspectiva cambia — de "¿qué parámetros maximizan estas métricas en el benchmark?" a "¿qué está señalando la caída de estas métricas sobre el estado del sistema en producción?".

Faithfulness en producción no solo mide si las respuestas son correctas — mide si el sistema está hallucinating. Una caída de faithfulness de 0.87 a 0.79 puede tener tres causas distintas con remedios distintos. La primera es una degradación del modelo LLM: si OpenAI actualizó GPT-4o y el nuevo comportamiento del modelo tiende menos a ceñirse al contexto proporcionado, la caída de faithfulness es sistémica y requiere evaluar si el prompt necesita ajustes para el nuevo comportamiento del modelo. La segunda es una degradación del retrieval: si context precision cayó simultáneamente (los chunks recuperados son menos relevantes), el LLM tiene menos contexto útil y "rellena" con conocimiento de preentrenamiento, produciendo afirmaciones sin soporte en el contexto. La tercera es un drift del corpus: los documentos en la base de conocimiento se volvieron obsoletos y ya no contienen la información que las queries buscan — el LLM no puede citar fuentes que no existen en el contexto recuperado. El análisis de las trazas de evaluación (qué afirmaciones específicas no tienen soporte en el contexto) y la correlación con otras métricas (¿cayó también context precision? ¿cayó el reranking score medio?) permite diagnosticar cuál de las tres causas aplica.

Answer Relevance en producción mide si el sistema está respondiendo las preguntas que los usuarios realmente hacen o si está derivando hacia respuestas tangencialmente relacionadas. Una caída sostenida de answer relevance puede indicar que el dominio de uso

evolució — los usuarios están formulando preguntas de un área nueva que la base de conocimiento no cubre bien, lo que lleva al sistema a responder sobre lo que tiene en lugar de sobre lo que se preguntó. También puede indicar un problema en el system prompt del agente si los cambios recientes al prompt modificaron las instrucciones de respuesta de formas que sesgan el contenido hacia ciertos tipos de respuesta.

Completeness es una métrica personalizada del proyecto que RAGAS estándar no incluye. Mide si la respuesta cubre todos los puntos clave que el anotador del golden dataset identificó como necesarios para responder correctamente la pregunta. La implementación usa un LLM evaluador (GPT-3.5-Turbo) que recibe la respuesta del sistema y la respuesta esperada del golden dataset, y clasifica qué puntos clave de la respuesta esperada están cubiertos: "punto sobre el tiempo de recovery: cubierto", "punto sobre el comando de rollback: no cubierto". El score es la proporción de puntos clave cubiertos. Completeness captura un tipo de degradación que faithfulness y answer relevance no detectan: el sistema puede generar respuestas faithfully ancladas en el contexto y relevantes para la pregunta pero que omiten información clave disponible en los documentos recuperados.

User Satisfaction se mide con feedback implícito del comportamiento real del usuario en producción. La tasa de continuación de conversación (el usuario envía un mensaje adicional en la misma sesión después de recibir la respuesta) es un proxy de satisfacción — los usuarios que obtienen la respuesta que necesitan frecuentemente continúan la conversación para profundizar; los usuarios insatisfechos cierran la sesión o reformulan la misma pregunta. La tasa de reformulación (el usuario repite la misma pregunta o una variante muy similar en la siguiente interacción) es un proxy de insatisfacción con la respuesta anterior. Estos señales son ruidosas individualmente pero estadísticamente significativas en ventanas de análisis de 24 horas con volumen suficiente de tráfico.

Métricas de evaluación de calidad en perspectiva operativa

- **Faithfulness:** en producción, una caída indica uno de tres problemas — drift de modelo LLM, degradación del retrieval (correlacionar con context_precision), o obsolescencia del corpus. Las trazas de evaluación identifican qué afirmaciones específicas no tienen soporte.
- **Answer Relevance:** en producción, una caída indica evolución del dominio de uso (los usuarios preguntan sobre áreas no cubiertas) o degradación del system prompt. Comparar con la distribución de queries de producción para diagnosticar.

- **Completeness personalizada:** LLM evaluador (GPT-3.5-Turbo) que clasifica cobertura de puntos clave del golden dataset; umbral de aceptación $\geq 75\%$; detecta respuestas correctas pero incompletas que faithfulness y answer relevance no capturan.
- **Context Precision:** en producción, mide la eficiencia del retrieval — si cae, el reranker está incluyendo chunks irrelevantes en el contexto; revisar threshold del reranker y distribución de reranking scores.
- **User Satisfaction:** session continuation rate (proxy de satisfacción positiva), reformulation rate (proxy de insatisfacción); calculados en ventanas de 24h sobre el tráfico completo, no sobre el 5% muestreado para RAGAS.

***Nota del Arquitecto:** La correlación entre métricas es la señal más valiosa del framework de evaluación — más que cualquier métrica individual. Una caída de faithfulness sola puede ser noise estadístico en la muestra del 5%. Una caída de faithfulness correlacionada con una caída de context_precision y un aumento del reranking score medio es una señal clara de degradación del retrieval. Una caída de faithfulness correlacionada con un aumento de answer_relevance es paradójica y señala que el sistema está siendo más "creativo" (menor fidelidad al contexto) pero más "relevante" (las respuestas creativas coinciden mejor con lo que los usuarios buscan) — lo que a su vez es una señal de que el golden dataset no representa bien la distribución de producción, porque los anotadores del dataset son más exigentes que los usuarios reales.*

Las métricas de calidad en producción producen un dataset de aprendizaje continuo: cada semana de evaluación online revela qué tipos de preguntas el sistema maneja mal, qué patrones de respuesta producen baja completeness y qué áreas del corpus tienen baja cobertura. Este dataset orienta las decisiones de mejora del Capítulo 10 con evidencia de producción real.

Para recordar: Las métricas automáticas de calidad como RAGAS son una aproximación al juicio humano en producción — su valor real está en la correlación entre métricas, que permite diagnosticar causas, no solo en los valores individuales.

Módulo 12 – Capítulo 07 – Sección 04

Evaluación de rendimiento: latencia end-to-end, throughput y costo por petición

La evaluación de rendimiento de un sistema RAG agéntico tiene una complejidad que los sistemas de software convencionales no tienen: la latencia no es una función simple de los recursos asignados sino la suma de las latencias de componentes externos no controlados por el equipo — la API de OpenAI, la API de Cohere, la instancia de Qdrant Cloud — más la latencia de los componentes internos. Optimizar la latencia requiere primero instrumentar cada etapa del pipeline para identificar el cuello de botella, porque las suposiciones sobre qué etapa es más lenta frecuentemente son incorrectas. Muchos equipos asumen que el cuello de botella es el LLM de generación y lo es — pero la segunda mayor contribuidora de latencia es frecuentemente el reranker de Cohere, no Qdrant, lo que cambia completamente la estrategia de optimización.

La instrumentación de latencia por etapa usa spans OpenTelemetry con tiempos de inicio y fin de cada componente. Los valores típicos observados en el sistema durante las pruebas de carga son: embedding de query (text-embedding-3-small): P50 75ms, P95 120ms; búsqueda en Qdrant (híbrida): P50 40ms, P95 80ms; reranking Cohere (top-20 candidatos): P50 280ms, P95 450ms; compresión de contexto (GPT-3.5-Turbo): P50 800ms, P95 1400ms; generación LLM (GPT-4o con 2400 tokens de input + 250 de output): P50 1200ms, P95 2000ms. La suma de las medianas es aproximadamente 2395ms — dentro del SLA de 3000ms P95, con margen para la varianza de cada componente. El percentil P95 end-to-end de 3150ms está ligeramente sobre el SLA, lo que llevó al equipo a reducir el timeout del compressor de contexto de GPT-3.5-Turbo y a activar el cache de embedding para queries recurrentes (reduciendo el tiempo de embedding de 75ms a < 5ms para queries con cache hit).

El benchmark de throughput usa Locust con tres escenarios de carga. El escenario de **baseline** simula 10 usuarios concurrentes enviando una query cada 2 segundos — este escenario establece el baseline de latencia sin contención. El escenario de **carga sostenida** simula 50 usuarios concurrentes con ramp-up de 30 segundos — este es el escenario del SLA, donde se miden los percentiles de producción. El escenario de **carga pico** simula 100

usuarios concurrentes durante 5 minutos — este escenario no es el SLA pero evalúa cómo el sistema se degrada bajo carga extrema y si el cluster autoscaler de EKS responde dentro de los 2 minutos esperados. Los resultados del benchmark con 50 usuarios concurrentes son: throughput de 23.4 req/s en steady state, P95 de 2.8s y error rate de 0.02% (principalmente timeouts de la API de OpenAI bajo carga).

La evaluación de costo por petición en producción usa las métricas de tokens consumidos por componente (disponibles en los responses de la API) convertidas a USD con las tarifas vigentes. El breakdown típico por petición con el sistema configurado según los ADRs: embedding query ($0.5 \text{ tokens} \times 0.00002 \text{ USD}/1000 = 0.00001 \text{ USD}$), reranking Cohere ($0.001 \text{ USD por } 1000 \text{ docs} \times 20 \text{ docs} = 0.00002 \text{ USD}$), compresión de contexto GPT-3.5-Turbo ($512 \text{ tokens input} + 200 \text{ output} \times \text{tarifas} = 0.0003 \text{ USD}$), generación GPT-4o ($2400 \text{ tokens input} \times 0.005 \text{ USD}/1000 = 0.012 \text{ USD} + 250 \text{ tokens output} \times 0.015 \text{ USD}/1000 = 0.00375 \text{ USD}$). Total API cost: aproximadamente 0.0161 USD por petición típica, dentro del constraint de 0.02 USD. Las peticiones con múltiples iteraciones del agente (preguntas complejas que requieren 3-5 llamadas a herramientas) pueden llegar a 0.035 USD — por encima del constraint — lo que llevó al equipo a añadir un prompt de advertencia para queries clasificadas como "alta complejidad" antes de ejecutarlas.

Métricas de evaluación de rendimiento

- **Latencia por etapa (P50 / P95):** embedding 75/120ms, Qdrant search 40/80ms, Cohere rerank 280/450ms, context compressor 800/1400ms, LLM generation 1200/2000ms — instrumentados con spans OpenTelemetry en Grafana Tempo.
- **Latencia end-to-end:** P50 < 2.4s, P95 < 3s bajo 50 usuarios concurrentes con ramp-up de 30s — SLA verificado en benchmark Locust semanal.
- **Throughput:** 23.4 req/s en steady state con 50 usuarios concurrentes; benchmark Locust con escenarios 10/50/100 usuarios para caracterizar la curva de degradación.
- **Costo por petición:** breakdown por componente (embedding + reranking + compresión + generación) = 0.0161 USD típico; outliers de alta complejidad hasta 0.035 USD con alerta de "alta complejidad" al usuario; atribución por equipo para chargeback interno.
- **Costo proyectado mensual:** $\text{costo_API} \times \text{volumen_mensual_estimado} = 0.0161 \times 5000 \text{ queries/día} \times 30 \text{ días} \approx 2400 \text{ USD/mes}$; breakdown separado de costos de infraestructura (EKS + RDS + Redis + Qdrant Cloud $\approx 800 \text{ USD/mes}$).

Nota del Arquitecto: La distribución de costo por complejidad de query es una señal operativa importante que los equipos frecuentemente no monitorizan. En el benchmark de producción, el 5% de las queries más complejas (las que requieren 4-5 iteraciones del agente) generan el 25% del costo total. Identificar esas queries complejas y analizar si son preguntas genuinamente difíciles (que justifican el costo) o preguntas que el agente maneja ineficientemente (que requieren mejora del razonamiento) puede reducir el costo total en un 15-20% sin degradar la calidad para el 95% de los usuarios. El breakdown de costo por número de iteraciones del agente, disponible en las trazas de OpenTelemetry, es el análisis que permite tomar esa decisión con datos.

La evaluación de rendimiento no es un ejercicio puntual antes del primer despliegue — es una práctica semanal que detecta regresiones de rendimiento antes de que afecten al SLA. El Capítulo 7 continúa con la evaluación de seguridad: el cierre del ciclo que vincula el red teaming del Capítulo 5 con las métricas de seguridad del sistema en producción.

Para recordar: La latencia de un sistema RAG agéntico es la suma de latencias de sus componentes — el cuello de botella típico es la generación LLM, pero el reranking con modelos externos puede ser el segundo mayor contribuidor y debe incluirse en el análisis de rendimiento.

Módulo 12 – Capítulo 07 – Sección 05

Evaluación de seguridad: resultados del red teaming y métricas de efectividad de controles

La evaluación de seguridad del sistema no termina con la sesión de red teaming del Capítulo 5 — esa sesión establece el baseline. En producción, los controles de seguridad deben monitorearse continuamente: la tasa de rechazos por injection detectada, la tasa de intentos de evasión de autorización, los patrones de queries clasificadas como `suspicious` por el clasificador de intent. Estas métricas no miden la efectividad de los controles de forma directa (un atacante sofisticado no dispara las alertas de los controles que evade), pero miden el volumen y los patrones del intento de ataque que el sistema enfrenta en producción real.

Los resultados del red teaming del Capítulo 5 se presentan aquí no como un informe retrospectivo sino como el baseline de métricas de seguridad contra el que se mide el sistema en producción. La tasa de bypass global fue del 2% (1/50 ataques) después de las correcciones aplicadas — esto es la métrica de referencia. Si en producción la tasa de queries clasificadas como `malicious` y rechazadas aumenta significativamente, puede indicar que los atacantes están probando nuevas variantes; si la tasa de rechazos cae a cero durante semanas, puede indicar que el clasificador de intent tiene un error de calibración o que el tráfico de prueba que el equipo genera para testear los controles no está llegando al sistema.

La **matriz de resultados del red teaming** organiza los 50 ataques por categoría y resultado: prompt injection directa (14/15 mitigados, 1 bypass parcial en variante japonesa — corregido con clasificador multilingüe), prompt injection indirecta (13/15 mitigados, 2 bypasses en instrucciones en bloques de código Python — corregido añadiendo análisis de contenido de bloques de código en el sanitizador), evasión de autorización (10/10 mitigados — el filtro estructural de Qdrant bloqueó todos los casos), y DoS agéntico (9/10 mitigados después de correcciones — el caso restante es una variante de baja probabilidad documentada como riesgo residual aceptado en el ADR-004). La tasa global post-correcciones es 2% (1/50), por debajo del objetivo de 5%.

Los **bypasss convertidos en regresiones** son el output más valioso del red teaming desde la perspectiva de la evaluación continua. Los 3 bypasses originales son ahora tests de regresión permanentes que se ejecutan antes de cada deploy que modifica los controles de seguridad: el test del ataque en japonés que evadió la lista negra, el primer test de instrucción en bloque de código Python y el segundo. Que estos tests pasen es condición necesaria para que el deploy avance — si algún cambio en el sistema rompe la mitigación de un bypass documentado, el test falla y bloquea el deploy.

Las **métricas de seguridad en producción** que el dashboard de Grafana monitorea continuamente son: tasa de rechazos por categoría (`injection_detected`, `rate_limit`, `auth_failure`, `schema_validation`) en ventanas de 1 hora, con alerta cuando `injection_detected` supera 10 rechazos/minuto (posible ataque coordinado); tasa de queries clasificadas como `suspicious` (threshold configurable, alerta cuando supera el 5% del tráfico total); y tasa de `allowed_document_types` violations (queries que el agente intentó satisfacer buscando tipos de documentos no autorizados, bloqueadas por el filtro de Qdrant). Esta última métrica a cero en operación normal confirma que el control de autorización a nivel de retrieval funciona correctamente.

Resultados del red teaming y métricas de seguridad

- **Prompt injection directa:** 14/15 mitigados post-correcciones (93.3% → 100% con clasificador multilingüe); test de regresión: variante en japonés permanente en la suite de adversarial tests.
- **Prompt injection indirecta:** 13/15 mitigados post-correcciones (86.7% → 100% con análisis de bloques de código); tests de regresión: 2 casos con instrucciones en comentarios Python permanentes en la suite.
- **Evasión de autorización:** 10/10 mitigados (100%) — el filtro estructural de Qdrant es la garantía; sin bypasses en ninguna variante evaluada.
- **DoS agéntico:** 9/10 mitigados post-correcciones (80% → 90%); 1 caso residual documentado en el ADR-004 como riesgo residual aceptado (requeriría WAF para mitigación completa).
- **Tasa global post-correcciones:** 1/50 (2%) — por debajo del objetivo de 5%; el único caso residual es el DoS de alta complejidad sin WAF.
- **Métricas de producción:** rechazos por `injection_detected` en Grafana, queries `suspicious` rate < 5% del tráfico total, `allowed_document_types` violations = 0 en operación normal.

Nota del Arquitecto: Una observación sobre el 100% de efectividad del filtro de autorización a nivel de Qdrant: este resultado no significa que el control sea perfecto contra todos los vectores posibles. Los 10 casos evaluados cubrieron las variantes más obvias de evasión de autorización; un atacante con acceso de API key y tiempo suficiente para experimentar puede encontrar variantes que el red teaming no anticipó. El 100% en el red teaming de 10 casos significa que el control funciona para los casos evaluados, no que es impenetrable. La práctica de expandir el red teaming con 5-10 nuevos casos de evasión de autorización en cada sesión trimestral es lo que produce mejora real de la seguridad, no la confianza en el resultado del último red teaming.

El framework de evaluación de seguridad, combinado con el monitoreo continuo de métricas de producción y el red teaming trimestral, convierte la seguridad de un estado puntual verificado antes del deploy en una práctica de mejora continua. El Capítulo 7 cierra con la síntesis del framework de evaluación end-to-end y su rol en el ciclo de mejora continua del sistema.

Para recordar: Los resultados del red teaming no son un informe de éxito o fracaso — son el mecanismo de mejora continua de la seguridad, donde cada bypass detectado es un control que debe mejorarse antes del deploy a producción.

Módulo 12 – Capítulo 07 – Sección 06

Cierre: la evaluación como infraestructura de aprendizaje continuo

El Capítulo 7 construyó el framework que convierte el sistema de IA en un sistema mejorable con evidencia. Sin él, el equipo opera con intuición — "parece que la calidad mejoró", "creo que la latencia aumentó". Con él, el equipo opera con datos — "faithfulness online cayó 0.05 puntos en los últimos 7 días, correlacionado con un incremento del 15% en el tiempo de first-token de GPT-4o según los spans de OpenTelemetry, posiblemente por una actualización del modelo del proveedor que aún no está documentada en el changelog de OpenAI". La diferencia entre esas dos posiciones no es solo de precisión — es de capacidad de acción: la primera no orienta ninguna decisión, la segunda tiene un diagnóstico preliminar y una hipótesis de causa.

El framework de evaluación continua también cierra la brecha entre el golden dataset estático y la realidad de producción. El golden dataset fue construido por ingenieros que anticiparon los tipos de preguntas que el sistema recibiría. La evaluación online sobre el 5% del tráfico real revela los tipos de preguntas que nadie anticipó: preguntas sobre incidentes recientes que no están en la documentación indexada, preguntas que mezclan terminología de dos dominios distintos de formas que el sistema no maneja bien, preguntas que requieren conocimiento sobre la versión específica del software que el usuario tiene instalado (que el agente no puede responder con certeza sin esa información). Cada uno de esos patrones de fallo es candidato a convertirse en nuevas entradas del golden dataset y en nuevos tipos de documentos a indexar.

La evaluación no es la fase final del ciclo de ingeniería de IA — es una fase permanente que coexiste con el desarrollo, el despliegue y la operación. El pipeline CI/CD del Capítulo 6 ya ejecuta evaluación RAGAS como gate de cada deploy; el framework de evaluación de este capítulo extiende esa práctica a la operación continua en producción. El ciclo completo es: evaluar → detectar degradación → diagnosticar causa → planificar mejora → implementar → evaluar de nuevo. Sin el primer paso, el ciclo no puede comenzar. Sin el último, no se puede saber si la mejora fue efectiva.

El Capítulo 8 construye sobre el framework de evaluación para implementar la observabilidad completa del sistema: las trazas distribuidas, el dashboard operativo y las alertas que convierten las métricas de evaluación en señales de operación en tiempo real.

Lo que el Capítulo 7 implementó

- **Framework tri-capa:** evaluación RAG (offline con golden dataset + online con 5% del tráfico real), evaluación agéntica (task completion, hallucination, iterations, tool accuracy) y evaluación de sistema (latencia, costo, error rate, disponibilidad) en correlación.
- **Golden dataset con IAA:** construcción con muestreo estratificado, anotación con confidence score, Inter-Annotator Agreement > 80% como criterio de calidad, división 80/20 train/test, proceso de mantenimiento documentado.
- **Evaluación de calidad operativa:** faithfulness, answer relevance y completeness personalizada en perspectiva de diagnóstico — qué causa la caída de cada métrica y cómo correlacionar las señales para identificar la causa raíz.
- **Evaluación de rendimiento:** latencia por etapa con spans OTel (valores típicos por componente), benchmark Locust 50 usuarios concurrentes, costo por petición desagregado con atribución por equipo y alertas de presupuesto.
- **Evaluación de seguridad:** resultados del red teaming como baseline operativo, bypasses convertidos en regresiones permanentes, métricas de producción de rechazos y queries suspicious en Grafana.

***Nota del Arquitecto:** El mayor error que he visto en frameworks de evaluación de IA es tratar el golden dataset como un artefacto que se construye una vez y se usa indefinidamente. El dominio de uso del sistema evoluciona: nuevos tipos de documentos, nuevas áreas de conocimiento, nuevos patrones de consulta de los usuarios. Un golden dataset que no evoluciona con el sistema produce métricas que se desconectan progresivamente de la calidad real en producción — el sistema puede tener faithfulness de 0.90 en el golden dataset de hace seis meses mientras la calidad en producción es 0.78 porque el dominio cambió. El proceso de mantenimiento del golden dataset — revisión trimestral, expansión con casos de producción, depreciación de preguntas obsoletas — debe ser tan parte del roadmap del equipo como el desarrollo de nuevas funcionalidades.*

La evaluación es la capacidad que convierte un sistema de IA en un sistema de ingeniería — sin métricas cuantificables, no hay forma de demostrar mejora, detectar degradación ni tomar

decisiones de arquitectura basadas en evidencia.

Para recordar: La evaluación es la capacidad que convierte un sistema de IA en un sistema de ingeniería — sin métricas cuantificables, no hay forma de demostrar mejora, detectar degradación ni tomar decisiones de arquitectura basadas en evidencia.

"Without data, you're just another person with an opinion." — W. Edwards Deming